

Lab 9: IPC Problems

1. Write a program in C to implement Peterson's Solution.

Code and Output:

```
#include <stdio.h>
#include <pthread.h>
#include <assert.h>
#include <sched.h>
int flag[2];
int turn;
const int MAX = 1e9;
int ans = 0;
void Pthread_mutex_lock(pthread_mutex_t *m) {
    int rc = pthread_mutex_lock(m);
    assert(rc == 0);
}
void Pthread_mutex_unlock(pthread_mutex_t *m) {
    int rc = pthread_mutex_unlock(m);
    assert(rc == 0);
}
void Pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                    void *(*start_routine)(void*), void *arg) {
    int rc = pthread_create(thread, attr, start_routine, arg);
    assert(rc == 0);
}
void Pthread_join(pthread_t thread, void **value_ptr) {
    int rc = pthread_join(thread, value_ptr);
    assert(rc == 0);
}
void lock_init() {
    flag[0] = flag[1] = 0;
    turn = 0;
}
void lock(int self) {
    flag[self] = 1;
    turn = 1 - self;
    while (flag[1 - self] == 1 && turn == 1 - self);
}
void unlock(int self) {
    flag[self] = 0;
}
void* func(void *s) {
    int i = 0;
    int self = (int)(long) s;
    printf("Thread Entered: %d\n", self);
```

```

        lock(self);
        for (i = 0; i < MAX; i++)
            ans++;
        unlock(self);
    }
    int main() {
        pthread_t p1, p2;
        lock_init();
        Pthread_create(&p1, NULL, func, (void*)0);
        Pthread_create(&p2, NULL, func, (void*)1);
        Pthread_join(p1, NULL);
        Pthread_join(p2, NULL);
        printf("Actual Count: %d | Expected Count: %d\n", ans, MAX * 2);
        return 0;
    }

```

```

Command Prompt x samir@NEXUS: ~/proces: x + v
samir@NEXUS:~$ ls
bash      demo.txt      if.sh         process      this         update_upgrade.sh
basic.sh.save  dir_in_loop.sh  lshw.html    save_log.sh  threads      user_input.sh
cube.sh      for.sh        mkdir_cmd.sh  snap         ubuntu       while.sh
samir@NEXUS:~$ cd process
samir@NEXUS:~/process$ nano mutex_example.c
samir@NEXUS:~/process$ gcc -o mutex_example mutex_example.c -lpthread
samir@NEXUS:~/process$ ./mutex_example
Thread Entered: 0
Thread Entered: 1
Actual Count: 2000000000 | Expected Count: 2000000000
samir@NEXUS:~/process$

```

2. Write a program in C to avoid race condition using Semaphores

Code and Output:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

#define KEY 0x1111

union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

```

```

struct sembuf p = { 0, -1, SEM_UNDO };
struct sembuf v = { 0, +1, SEM_UNDO };

int main() {
    int id = semget(KEY, 1, 0666 | IPC_CREAT);
    if (id < 0) {
        perror("semget");
        exit(11);
    }

    union semun u;
    u.val = 1;
    if (semctl(id, 0, SETVAL, u) < 0) {
        perror("semctl");
        exit(12);
    }

    int pid = fork();
    srand(pid);
    if (pid < 0) {
        perror("fork");
        exit(1);
    } else if (pid) {
        char *s = "abcdefgh";
        int l = strlen(s);
        for (int i = 0; i < l; ++i) {
            if (semop(id, &p, 1) < 0) {
                perror("semop p");
                exit(13);
            }
            putchar(s[i]);
            fflush(stdout);
            sleep(rand() % 2);
            putchar(s[i]);
            fflush(stdout);
            if (semop(id, &v, 1) < 0) {
                perror("semop v");
                exit(14);
            }
            sleep(rand() % 2);
        }
    } else {
        char *s = "ABCDEFGH";
        int l = strlen(s);
        for (int i = 0; i < l; ++i) {
            if (semop(id, &p, 1) < 0) {
                perror("semop p");
                exit(15);
            }
        }
    }
}

```

```

    }
    putchar(s[i]);
    fflush(stdout);
    sleep(rand() % 2);
    putchar(s[i]);
    fflush(stdout);
    if (semop(id, &v, 1) < 0) {
        perror("semop v");
        exit(16);
    }
    sleep(rand() % 2);
}
}
return 0;
}

```

```

Command Prompt  samir@NEXUS: ~/proces:  +  v
samir@NEXUS:~/process$ nano semaphore_example.c
samir@NEXUS:~/process$ gcc -o semaphore_example semaphore_example.c
samir@NEXUS:~/process$ ./semaphore_example
aaAAbbBBccCCddeDDEEffFFggGGhhHHsamir@NEXUS:~/process$

```